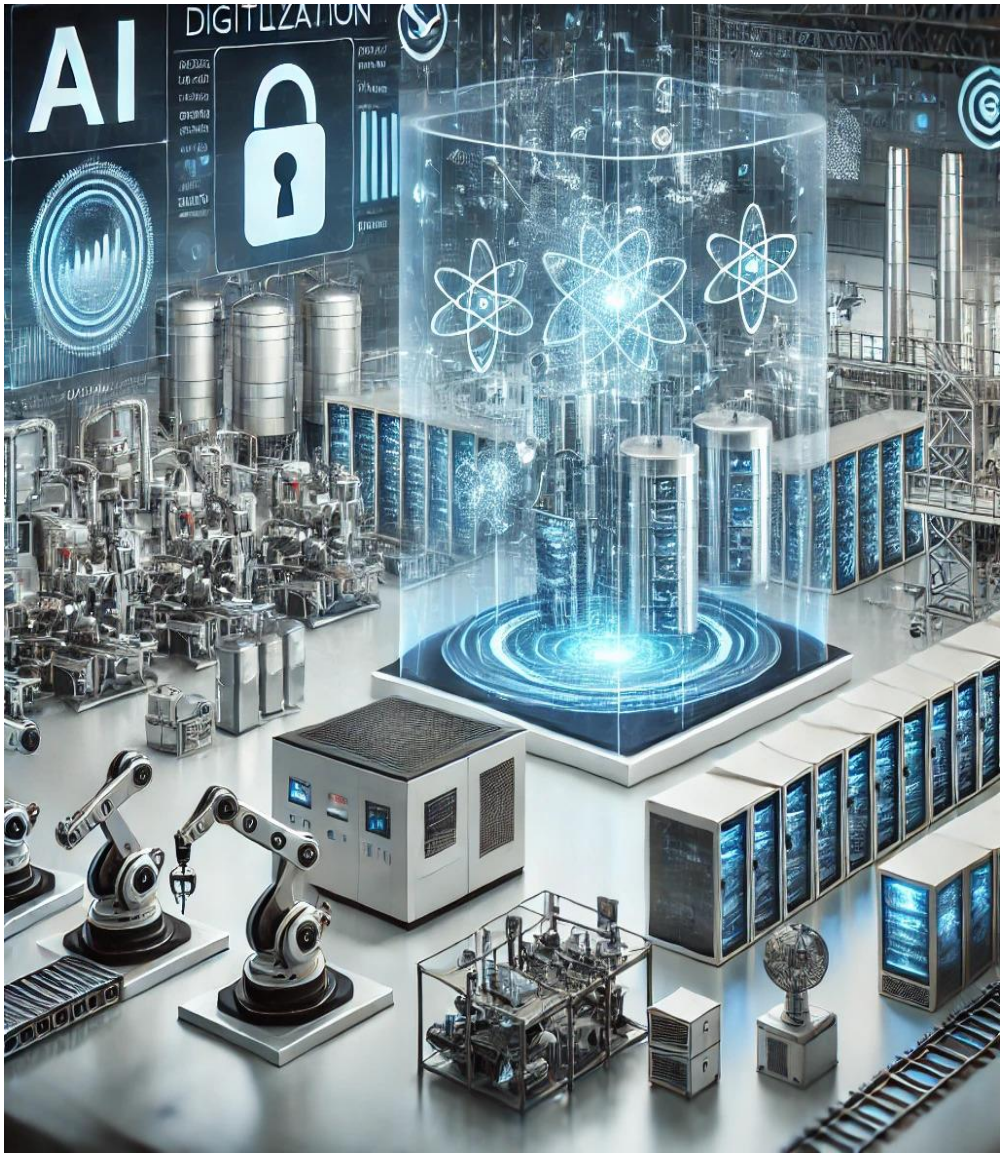
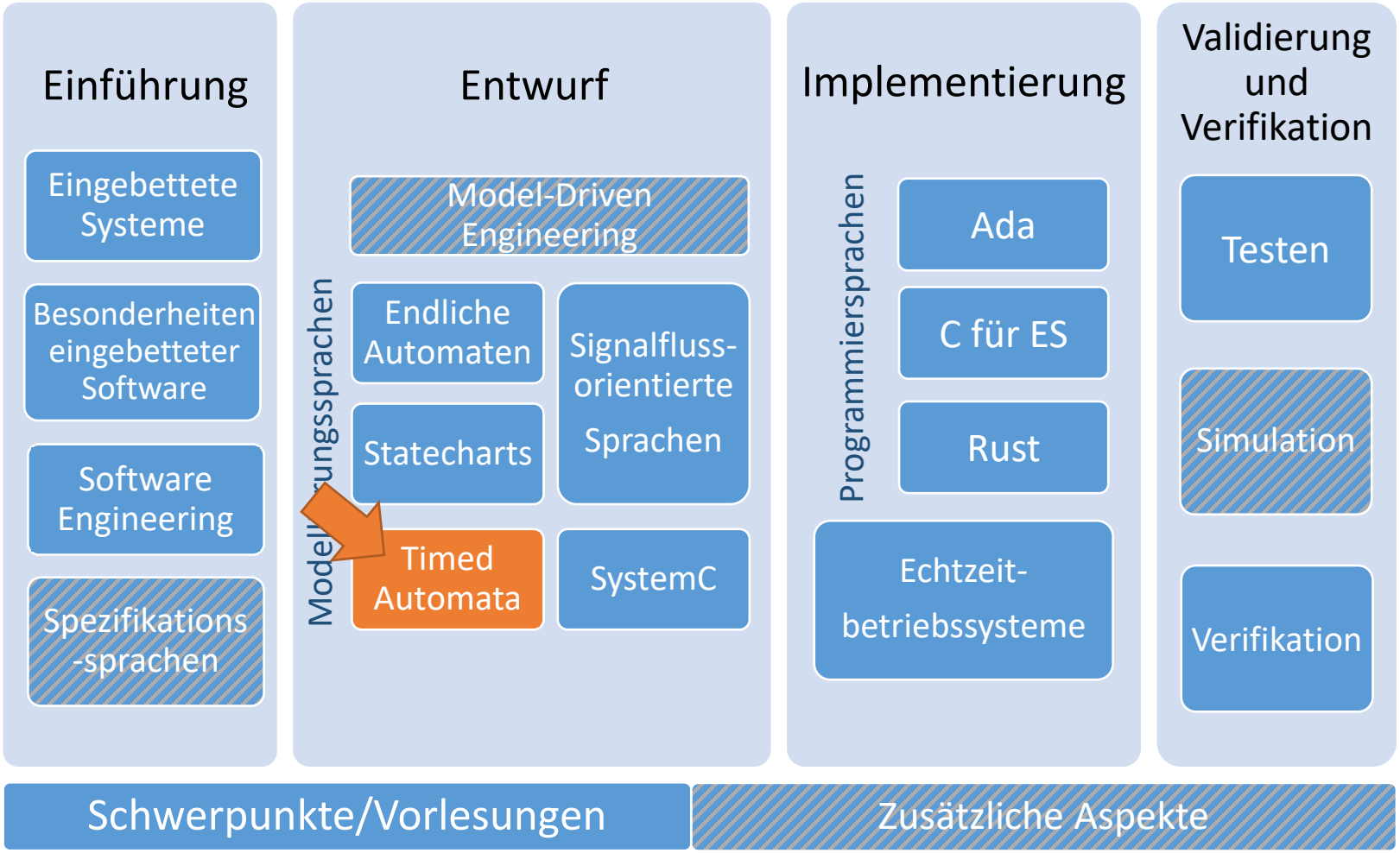


Hochschule für
Wirtschaft und Recht Berlin
Berlin School of Economics and Law

Embedded Systems Engineering
Timed Automata

Prof. Dr.-Ing. Sebastian Schlesinger
Professor of Business Computer Science
Cybersecurity and Embedded Systems Engineering

Diese Vorlesung



Inhalt

- Einführung
- Semantik
- Netzwerke von Timed Automata
- UPPAAL
- Fazit

Motivation

- FSMs können Hard- und Software formal spezifizieren, simulieren, verifizieren + besitzen intuitive grafische Darstellung
- Aber: berücksichtigen keine Zeit

- Zeitbehaftung + Echtzeit sind zentrale nichtfunktionale Eigenschaften von eingebetteten Systemen
- Daher: benötigen Modell, dass Zeit explizit berücksichtigt!

Timed Automata

Erweitern klassische endliche Automaten um Uhren:

- Uhren laufen **ständig** (kein Anhalten)
- Alle Uhren laufen mit der selben Geschwindigkeit (**perfekte** Uhren, **synchron** und **uniform**)
- Uhren können durch Transitionen auf 0 **zurückgesetzt** werden
- Uhren können das Schalten von Transitionen **beeinflussen** (Uhrenbedingungen)
- Wertebereich der Uhren: **positive reelle** Zahlen ($\mathbb{R}$+)

- Zeit als explizite, kontinuierliche Größe (reelle Werte)
- Grundlage für die Modellierung und Analyse von Systemen mit Echtzeitverhalten

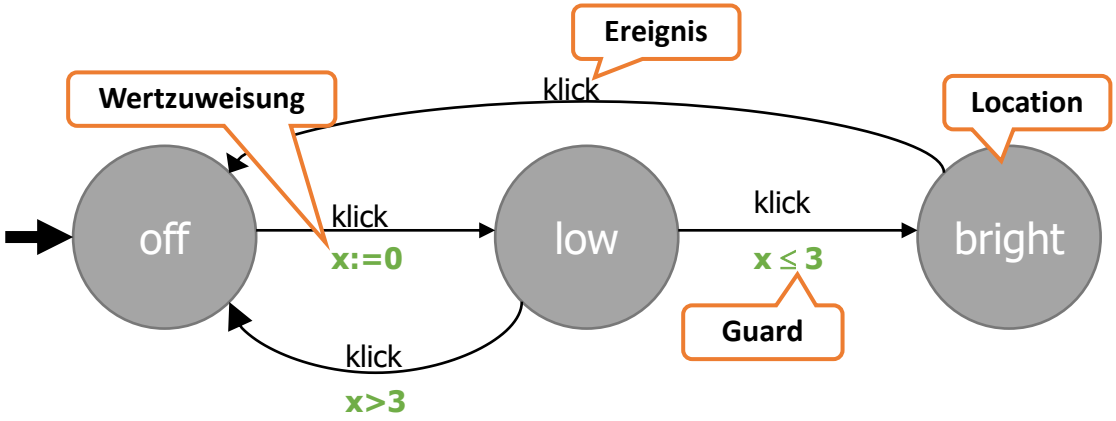
TA in der Praxis

- Bang & Olufsen: Audio- und Videoprotokolle
- LEGO® MindstormsTM
  - Control Program Synthesis
  - Verification of RCXTM Systems
- Philips: Audioprotokolle
- Verschiedene andere Protokolle:
  - Bounded Retransmission Protocol
  - Collision Avoidance Protocol
  - Commercial Field Bus Protocol
  - TDMA (Time Division Multiple Access) Protokoll (wird z.B. in FlexRay verwendet)
  - Fischer's Mutual Exclusion Protocol

Einführungsbeispiel

Doppelklick-Schalter:

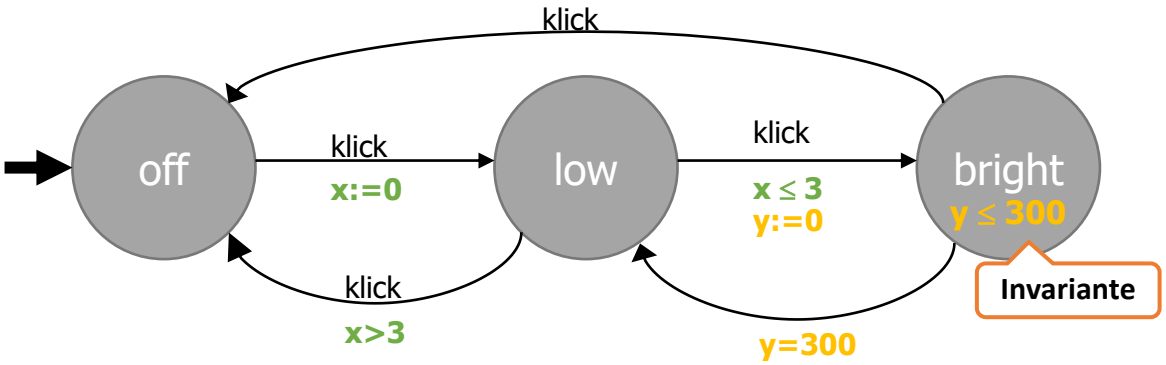
- Klick an, klick aus
- Wenn zweimal hintereinander innerhalb von 3 Zeiteinheiten geklickt wird, heller



- Eine Uhr x
- Bei Ereignis `klick`: Übergang zu `low`. Dabei wird Uhr x auf 0 zurückgesetzt.
- Bei erneutem `klick`:
  - Wenn innerhalb von 3 Zeiteinheiten: Übergang zu `bright`.
  - Sonst: Übergang zu `off`.

Einführungsbeispiel: Fortsetzung

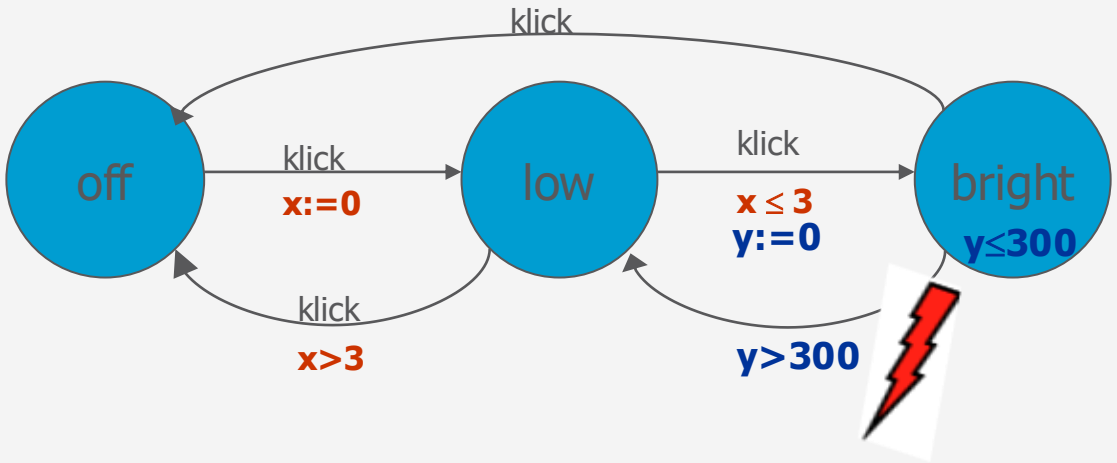
Zusätzliche Anforderung: nach *genau* 300 Zeiteinheiten wieder dunkler



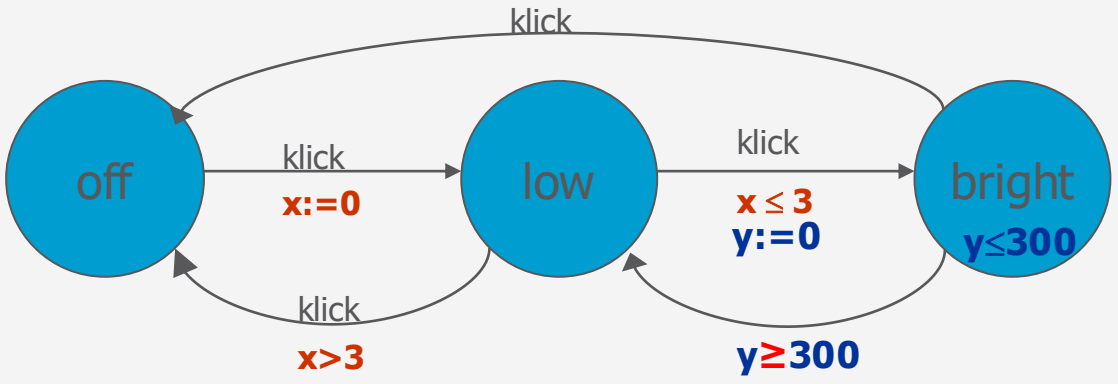
- Zusätzliche Uhr y
- In `bright` läuft eine Uhr:
  - Spätestens nach 300 Zeiteinheiten muss `bright` verlassen werden
  - 300 Zeiteinheit später ist der Übergang zu `low` möglich

Was wäre, wenn der Übergang stattdessen <u>spätestens</u> nach 300 Zeiteinheiten stattfinden soll?

Anwendungsbeispiel



Anwendungsbeispiel



Uhren

- Uhren werden durch Variablen x, y, z, … aus einer Menge C repräsentiert
- Eine Uhrenbelegung v ordnet jeder Uhr in C eine nichtnegative reelle Zahl zu, d.h. $v: C \rightarrow \mathbb{R}^+$
- VC: Menge der Uhrenbelegungen in C

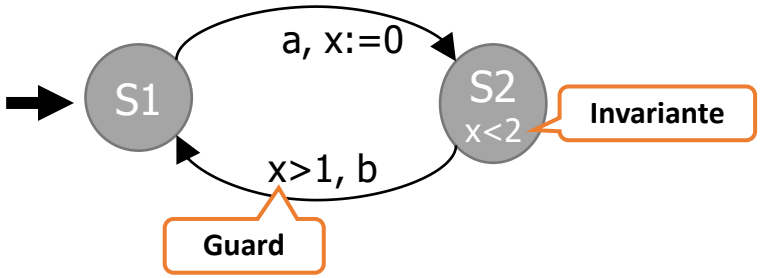
Uhrenbedingungen

Zwei Arten:

An Transitionen: Guards („Wächter“)
Transitionen können nur stattfinden, wenn die aktuelle Uhrenbelegung den Guard erfüllt (enable progress)

An Zuständen: Invarianten

- Der Automat kann nur in einem Zustand verweilen, so lange die Invariante erfüllt ist (ensure progress)
- Die Verletzung einer Invariante kann ggf. nur durch einen Zustandsübergang verhindert werden



Selbe Syntax:

Konjunktion von atomaren Vergleichsaussagen der Form

$x \simeq d$ oder $x - y \simeq d$

wobei d **ganzzahlig** und $\simeq$ eine Vergleichsoperation aus {<, ≤, =, ≥, >}

- $\Phi(C)$: Menge der Uhrenbedingungen über der Uhrenmenge C

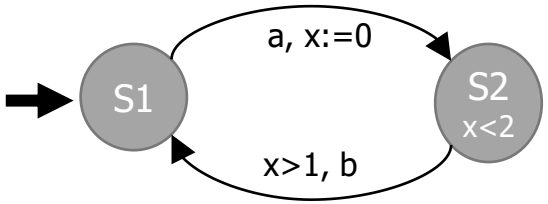
Syntax

Ein Timed Automaton ist ein Tupel

$$TA = (L, l_0, E \cup \{\tau\}, C, \mathrm{Inv}, T)$$

mit

- L – (endliche) Menge von Kontrollzuständen (*locations*, Stellen)
- $l_0 \in L$ – Anfangszustand
- E – endliches Alphabet von Ereignissen,
  τ ist spezielles internes Ereignis
- C – endliche Menge von Uhren
- $\mathrm{Inv}: L \to \Phi(C)$ – ordnet jedem Kontrollzustand eine Invariante zu
- T – Transitionsrelation (Zustandsübergänge)



Beispiel:

- $L = \{s_1, s_2\}$
- $l_0 = s_1$
- $E = \{a, b, \tau\}$
- $C = \{x\}$

- Invarianten:

s_1	wahr
s_2	$x < 2$

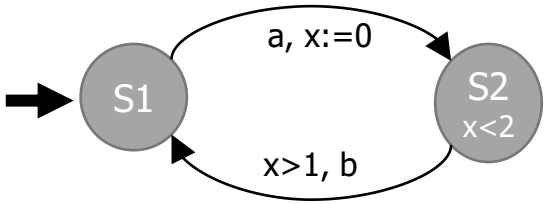
Transitionsrelation

Die Transitionsrelation T eines TA ist definiert als

$$T \subseteq L \times (E \cup \{\tau\}) \times \Phi(C) \times \Gamma(C) \times L$$

Ein Element $t = (l_1, e, g, r, l_2)$ bezeichnet

- Eine Transition von l_1 nach l_2
- mit Ereignis e
- einer Uhrenbedingung g (Guard)
- und einer Menge von Wertzuweisungen zu Uhren r (Zurücksetzen von Uhren)



Beispiel:

Transitionen:

$$s_1 \xrightarrow{a, wahr, x:=0} s_2$$

$$s_2 \xrightarrow{b, x>1, \emptyset} s_1$$

Erweiterungen

Die formale Definition von TA **enthält nur die Essenz** von echtzeitfähigen zustandsbasierten Modellen.

Wichtig, um Zeiteigenschaften zu verstehen und Methoden zur Verifikation zu entwickeln.

In der Praxis wird das Modell um verschiedene Konzepte **erweitert:**

- Parallele Komposition
- Kommunikation
- Synchronisation
- Variablen

Zunächst wollen wir aber das **Zeitverhalten gründlich verstehen**.

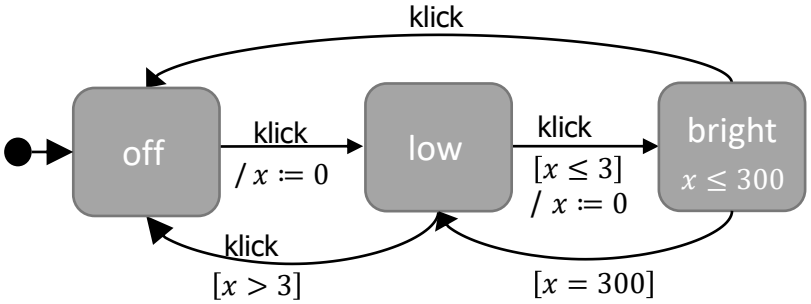
Inhalt

- Einführung
- Semantik
- Netzwerke von Timed Automata
- UPPAAL
- Fazit

Verhalten von Timed Automata

Müssen Verhalten präzise beschreiben. Beispiele:

- Welche Übergänge sind im Kontrollzustand `low` möglich?
  Es kommt auf die Uhrenbelegung an! Der Kontrollzustand allein ist eine unvollständige Beschreibung des Automatenzustandes.
- Kann im Zustand `bright` Zeit vergehen?
  Ja, solange die Invariante nicht verletzt wird.
- Kann während des Übergangs von `off` nach `low` Zeit vergehen?
  Zeit vergeht nur in Zuständen, sonst wäre die Uhrbelegung nicht eindeutig (ist die Zeit vor oder nach der Zurücksetzung von x vergangen?).

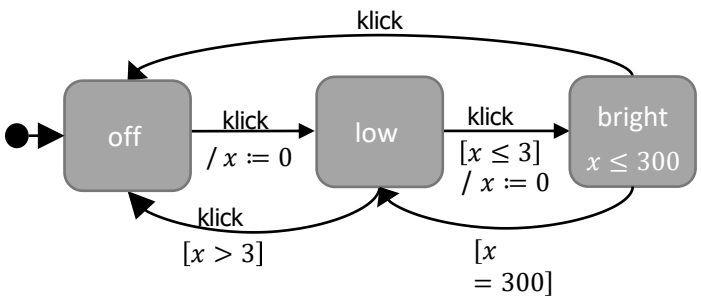


Semantik: Transitionssystem

Die TA Semantik bestimmt ein **Transitionssytem** aus **Konfigurationen** + **Übergängen:**

- Eine **Konfiguration** $(\boldsymbol{l}, \boldsymbol{v})$ besteht aus Kontrollzustand l und Uhrenbelegung v
  - Die Initialkonfiguration $(\boldsymbol{l_0}, \boldsymbol{v_0})$ besteht aus dem Initialzustand l_0 und der Uhrenbelegung v_0, in der alle Uhren auf 0 gestellt sind.
- Es gibt zwei Arten von Übergängen: **Zeitschritte** und **diskrete Transitionen**

Timed Automaton:



Transitionssystem:

(off; 0)

Semantik: Transitionssystem

Die TA Semantik bestimmt ein **Transitionssytem** aus **Konfigurationen** + **Übergängen:**

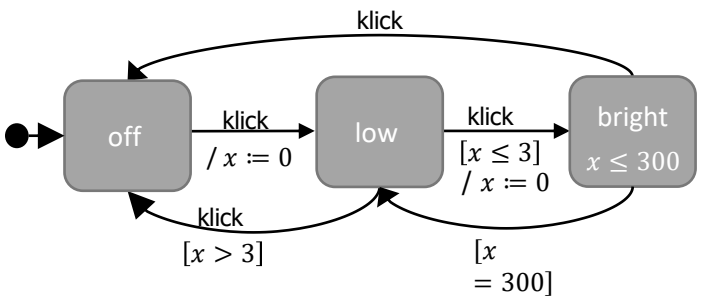
- Es gibt zwei Arten von Übergängen: **Zeitschritte** und **diskrete Transitionen**

Zeitschritt (timed step, delay)

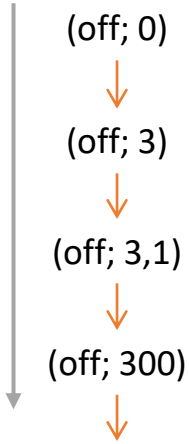
$$(l, v) \xrightarrow{\delta} (l, v + \delta)$$

- Kontrollzustand bleibt gleich
- Werte aller Uhren werden uniform um ein positives δ erhöht

Timed Automaton:



Transitionssystem:



Semantik: Transitionssystem

Die TA Semantik bestimmt ein **Transitionssytem** aus **Konfigurationen** + **Übergängen:**

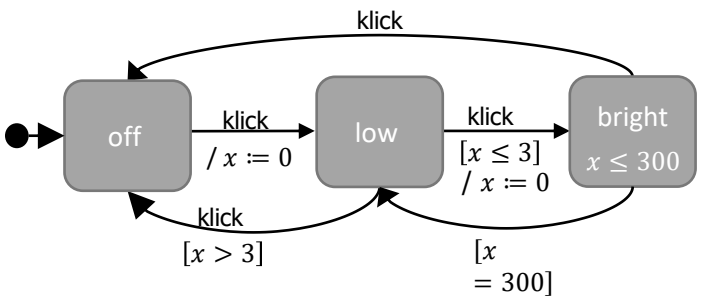
- Es gibt zwei Arten von Übergängen: **Zeitschritte** und **diskrete Transitionen**

Diskrete Transition

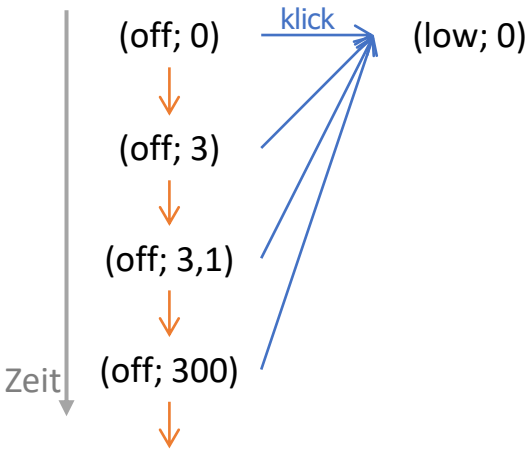
$$(l, v) \xrightarrow{e} (l', v')$$

- Entsprechend einem Element $l \xrightarrow{e,g,r} l'$ der Transitionsrelation (entlang einer Kante in der grafischen Darstellung)
- Uhrbelegung v muss den Guard g der Transition erfüllen
- Neue Uhrbelegung v' enthält zurückgesetzte Uhren

Timed Automaton:



Transitionssystem:



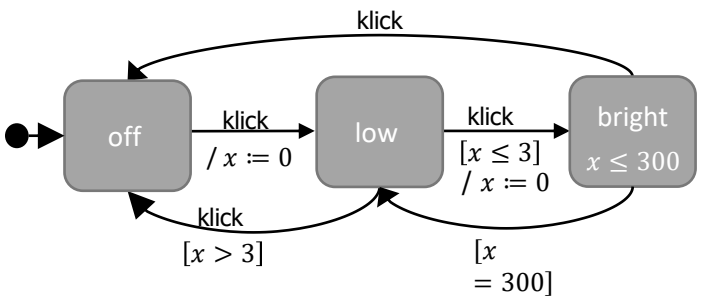
Semantik: Transitionssystem

Die TA Semantik bestimmt ein **Transitionssytem** aus **Konfigurationen** + **Übergängen:**

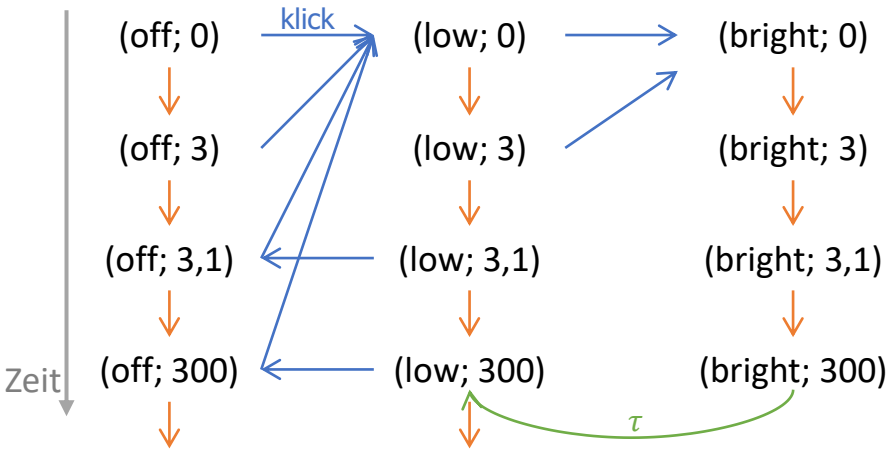
- Es gibt zwei Arten von Übergängen: **Zeitschritte** und **diskrete Transitionen**

- Zeitschritte und diskrete Transitionen werden beliebig hintereinander ausgeführt.
  - Allerdings nur möglich, wenn **keine Invariante verletzt** wird!
  - Beide Arten von Übergang sind verboten, wenn im erreichten Zustand eine Invariante verletzt wird.

Timed Automaton:



Transitionssystem:



Semantik: Pfade

- Ein Transitionssystem bestimmt eine Menge **möglicher Pfade**.

- Ein **Pfad** ist eine endliche oder unendliche Sequenz von Transitionen und Konfigurationen:

$$(l_0, v_0) \xrightarrow{a_0} (l_1, v_1) \xrightarrow{a_1} (l_2, v_2) \xrightarrow{a_2} (l_3, v_3) \xrightarrow{a_3} \ldots$$

mit

- (l_0, v_0) ist die initiale Konfiguration
- Jedes a_i ist eine diskrete Transition oder ein Zeitschritt

Inhalt

- Einführung
- Semantik
- Netzwerke von Timed Automata
- UPPAAL
- Fazit

Netzwerke von Timed Automata

Netzwerk = mehrere TA, die parallel laufen und miteinander kommunizieren

Die Ereignismenge der TA wird partitioniert in

- $c?$ Eingaben, passend zu…
- $c!$ …Ausgaben, und
- τ interne Ereignisse

- Jede Transition darf nur mit höchstens einer Eingabe <u>oder</u> einer Ausgabe beschriftet sein

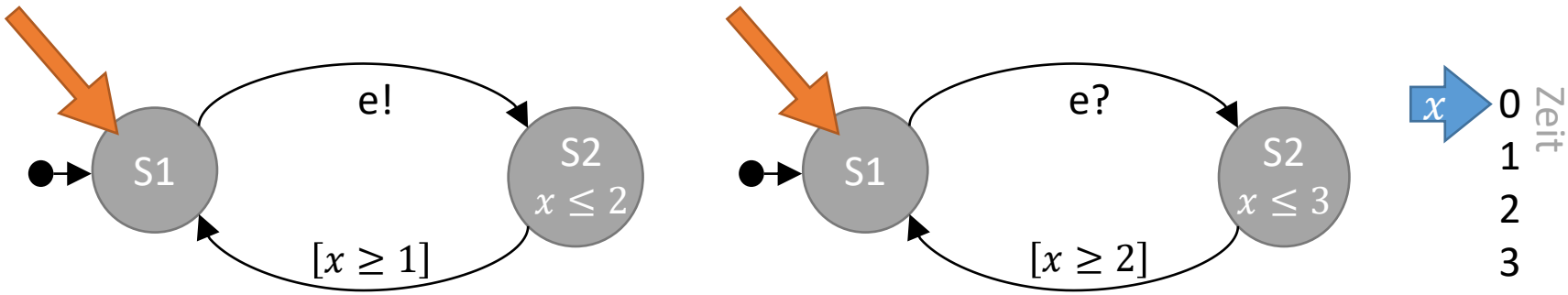
Parallele Ausführung

- **Synchroner Zeitverlauf** (synchrone, uniforme Uhren): alle Uhren laufen gleichmäßig
- **Verschachtelte diskrete Schritte** (*interleaving*):
  - Nicht-synchronisierte Schritte jedes Automaten werden einzeln ausgeführt
  - Übergang eines einzelnen Automaten
- **Synchronisation**: Ein- und Ausgabeereignisse: synchronisierte Schritte werden gleichzeitig von beiden Automaten ausgeführt
- **Kommunikation**: gemeinsame Variablen (in UPPAAL)

Binäre Synchronisation

Analogie: Telefonieren

- Sender: e! (Ausgabeereignis)
- Empfänger: e? (Eingabeereignis)

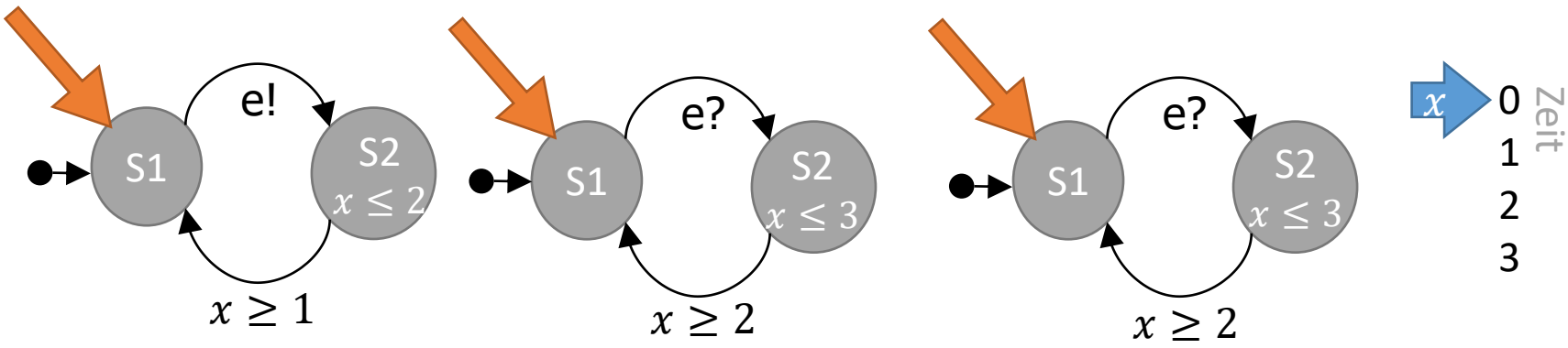


- Blockierend!
- Wenn ein Kommunikationspartner nicht empfangsbereit ist, muss der andere warten ⇨ e! und e? schalten gleichzeitig
- Bei mehreren möglichen Kommunikationspartnern: paarweise Synchronisation

Broadcast Synchronisation

Analogie: Radio, Präsenz-Vorlesung

- Spezieller Synchronisationsmechanismus in UPPAAL
- Ein Sender kann an mehrere Empfänger senden



- Nicht-blockierend!
- Der Sender muss nicht warten
- Alle empfangsbereiten Empfänger synchronisieren mit dem Sender, alle anderen “verpassen” die Nachricht

Parallelkomposition

Zwei Automaten können in einem kombiniert werden, mit dem gleichen Verhalten wie eine parallele Ausführung:

- Kontrollzustände sind **Paare der originalen Kontrollzustände**
- Initialzustand ist das Paar der originalen Initialzustände
- Produkt ist ein **geschlossenes System**, hat nur das **interne Ereignis τ**
- Die **Uhren sind vereinigt**, Uhren mit gleichen Namen teilen beide Automaten
- Invarianten sind zusammengesetzt
- **Interne Transitionen** der Teilautomaten sind **verschachtelt**
- Andere Transitionen sind **synchronisiert**

Außer es gibt weitere Kommunikationspartner im System

Uhren sind global. Bei UPPAAL gibt es auch lokale Uhren, diese werden nicht geteilt.

Parallelkomposition

Das **Produkt** zweier Timed Automata TA_1, TA_2 ist definiert als

$$TA_1 \,||\, TA_2 := \; (L_1 \times L_2, (L_{0,1}, L_{0,2}), \{\tau\}, C_1 \;\cup\; C_2, \mathrm{Inv}, T)$$

mit $\mathrm{Inv}(s_1, s_2) \;=\; \mathrm{Inv}(s_1) \;\wedge\; \mathrm{Inv}(s_2)$

Zusammengesetzte Transitionen:

- **Synchronisation**: Sender in TA_1 und Empfänger in TA_2 (oder umgekehrt)
- **Verschachtelung**:
  - Interner Schritt in TA_1: TA_1 arbeitet unabhängig
  - Interner Schritt in TA_2: TA_2 arbeitet unabhängig

Parallelkomposition: Transitionen

Synchronisation:

Für $l_1 \xrightarrow{c!,g_1,r_1} l_1'$ in TA_1 und $l_2 \xrightarrow{c?,g_2,r_2} l_2'$ in TA_2

(oder $l_1 \xrightarrow{c?,g_1,r_1} l_1'$ in TA_1 und $l_2 \xrightarrow{c!,g_2,r_2} l_2'$ in TA_2)

$$\mathrm{t} \coloneqq (l_1, l_2) \xrightarrow{\tau, g_1 \wedge g_2, r_1 \cup r_2} (l_1', l_2')$$

Verschachtelung:

Für $l_1 \xrightarrow{\tau,g_1,r_1} l_1'$ in TA_1: $\mathrm{t} \coloneqq (l_1, l_2) \xrightarrow{\tau,g_1,r_1} (l_1', l_2\ )$

TA_1 arbeitet unabhängig

Für $l_2 \xrightarrow{\tau,g_2,r_2} l_2'$ in TA_2: $\mathrm{t} \coloneqq (l_1, l_2) \xrightarrow{\tau,g_2,r_2} (l_1, l_2'\ )$

TA_2 arbeitet unabhängig

Parallelkomposition: *n* Automaten

Das **Produkt** von n Timed Automata ist definiert als

$$TA_1 || \dots || TA_n :=$$
$$\left(L_1 \times \cdots \times L_n, (L_{0,1}, \dots, L_{0,n}), \{\tau\}, C_1 \cup \cdots \cup C_n, \mathrm{Inv}(l_1, \dots, l_n), T\right)$$

mit $\mathrm{Inv}(l_1, \dots, l_n) = \mathrm{Inv}(l_1) \wedge \cdots \wedge \mathrm{Inv}(l_n)$

Zusammengesetzte Transitionen:

- für $i \neq j$ und $l_i \xrightarrow{a!, g_i, r_i} l_i'$ und $l_j \xrightarrow{a?, g_j, r_j} l_j'$

$$\boldsymbol{t} := \quad \boldsymbol{l} \xrightarrow{\boldsymbol{\tau}, \boldsymbol{g_i} \wedge \boldsymbol{g_j}, \boldsymbol{r_i} \cup \boldsymbol{r_j}} \boldsymbol{l}\left[\boldsymbol{l_i'} \backslash \boldsymbol{l_i}\right]\left[\boldsymbol{l_j'} \backslash \boldsymbol{l_j}\right]$$

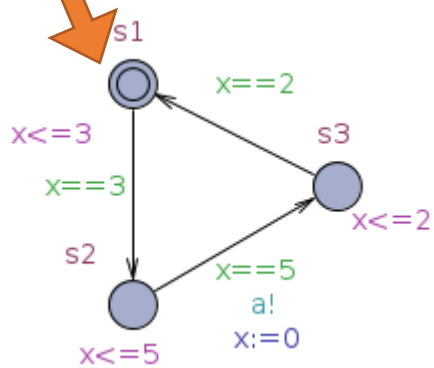
- für $l_i \xrightarrow{\tau, g, r} l_i'$

$$\boldsymbol{t} := \quad \boldsymbol{l} \xrightarrow{\boldsymbol{\tau}, \boldsymbol{g}, \boldsymbol{r}} \boldsymbol{l}[\boldsymbol{l_i'} \backslash \boldsymbol{l_i}]$$

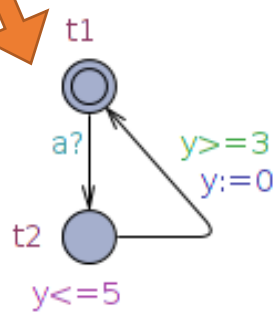
l ist ein Vektor von Kontrollzuständen $l = (l_1, \dots l_n)$ und $l[l_i' \backslash l_i]$ beschreibt, dass in Vektor l an der i-ten Stelle l_i durch l_i' ersetzt wird.

Beispiel: Parallelkomposition

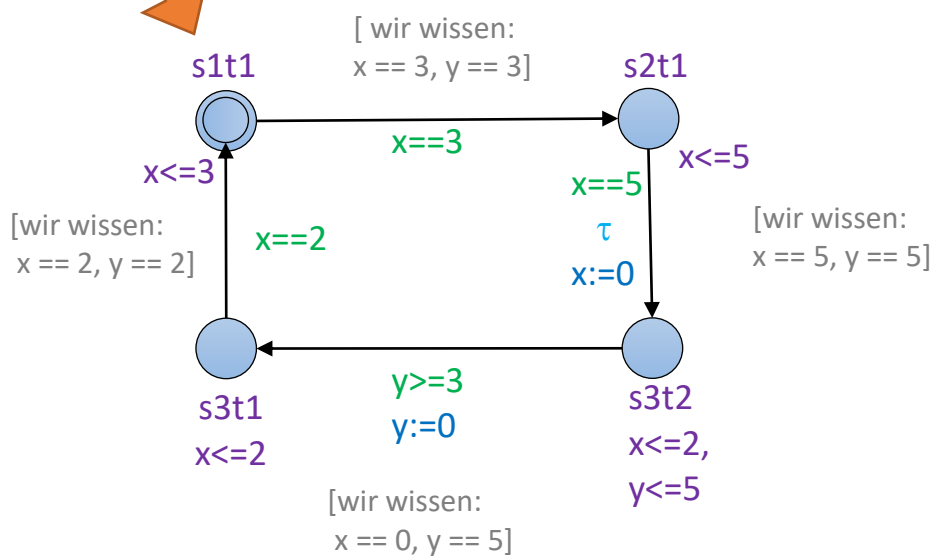
Automat 1



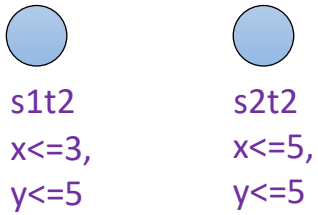
Automat 2



Parallelkomposition

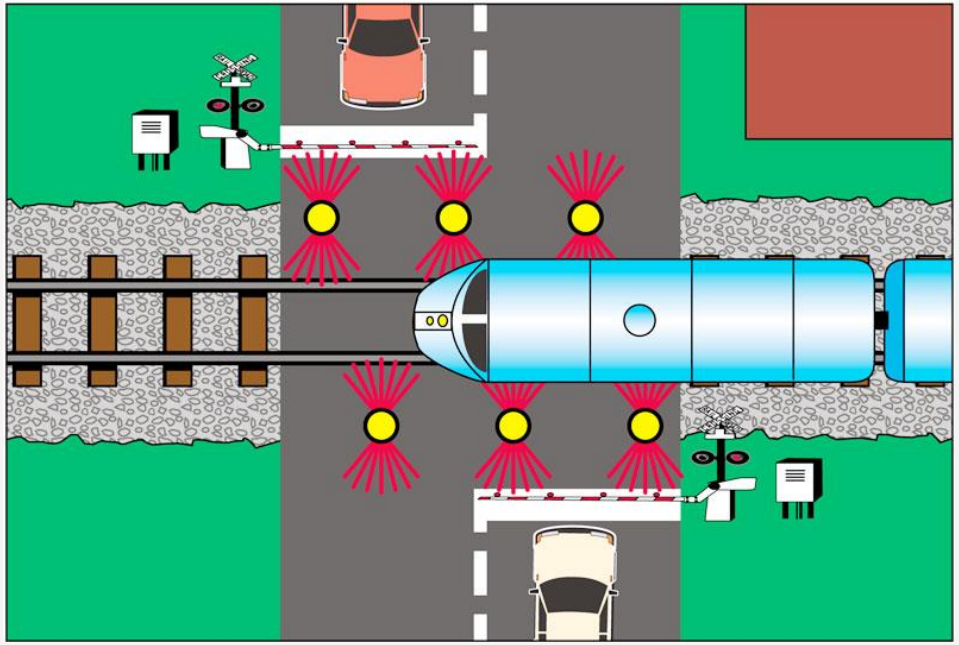


Nicht erreichbare zusammengesetzte Stellen:



Beispiel: Bahnübergang

⇨ kann als nebenläufige (parallele) Komposition von 3 Timed Automata modelliert werden

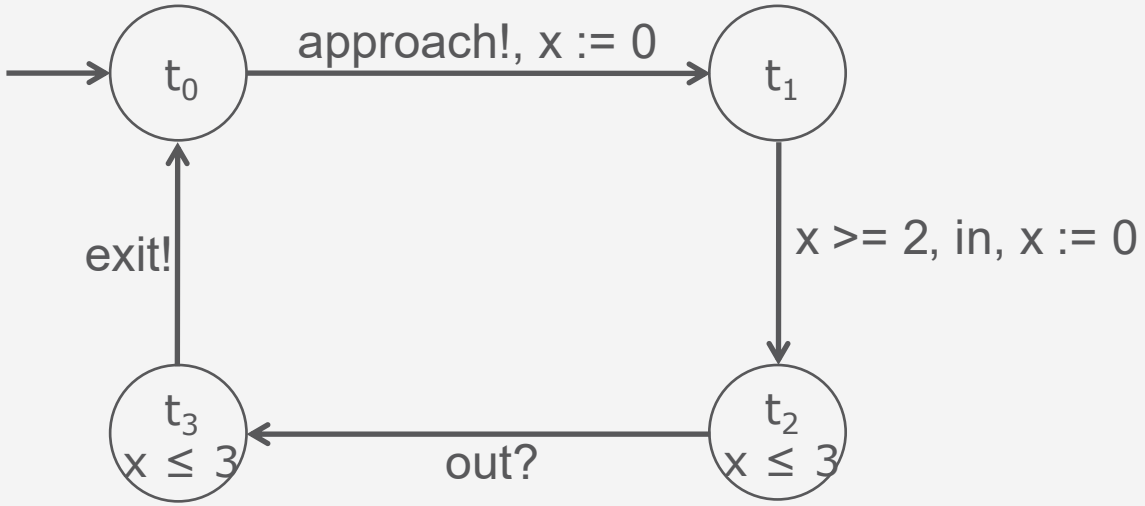


„Bahnübergang“
besteht aus drei Komponenten:

- Zug (*train*)
- Schranke (*gate*)
- Kontrolleinheit (*controller*)

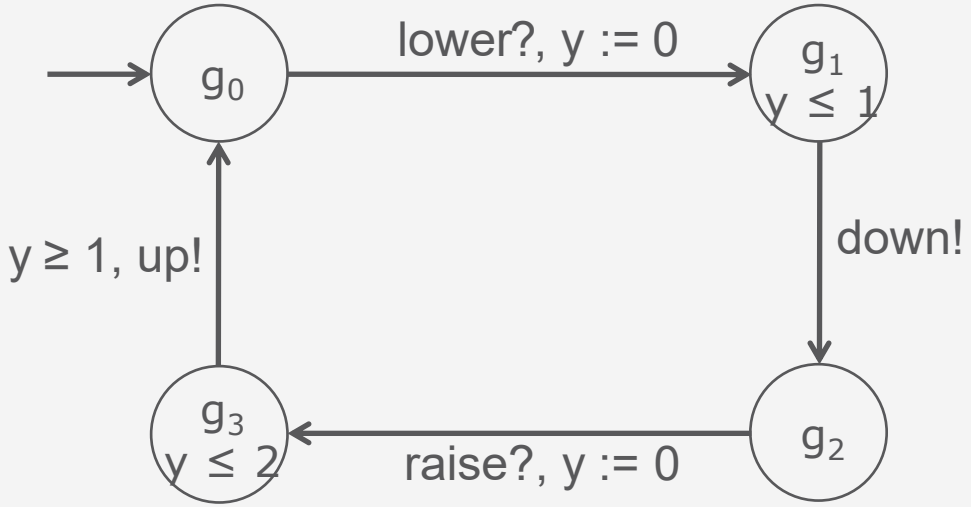
Zug als TA

- Zug (*train*) sendet mindestens 2 Zeiteinheiten (ZE) vor Erreichen des Bahnübergangs das Signal *approach* an die Kontrolleinheit (*controller*), bleibt maximal 3 ZE im Übergang und sendet das Signal *exit* beim Verlassen



Schranke als TA

- Öffnet und schließt wenn Signal kommt
- Schranke (*gate*) benötigt maximal 1 ZE für das Schließen, 1-2 ZE für das Öffnen



Kontrolleinheit als TA

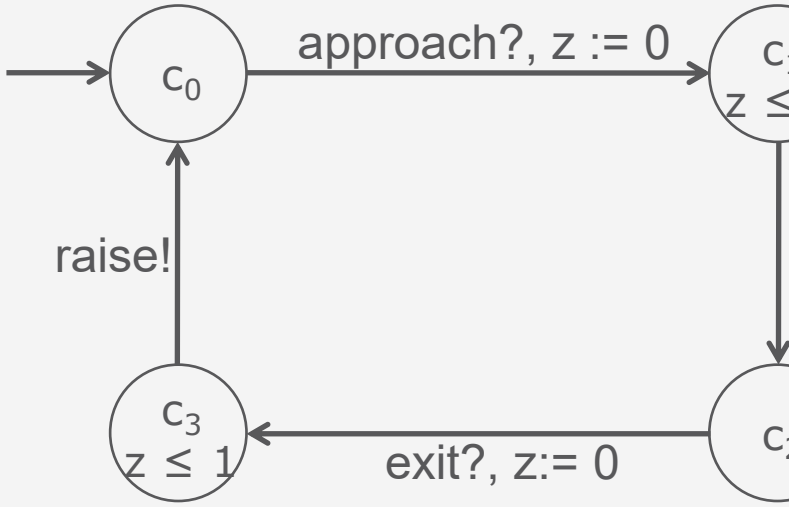
- Kontrolleinheit (*controller*) sendet genau 1 ZE nach Erhalt des Signals *approach* das Kommando *lower* an die Schranke (*gate*) und das Kommando *raise* nicht mehr als 1 ZE nach Erhalt des Signals *exit*

Modellieren Sie die Kontrolleinheit als Timed Automaton!

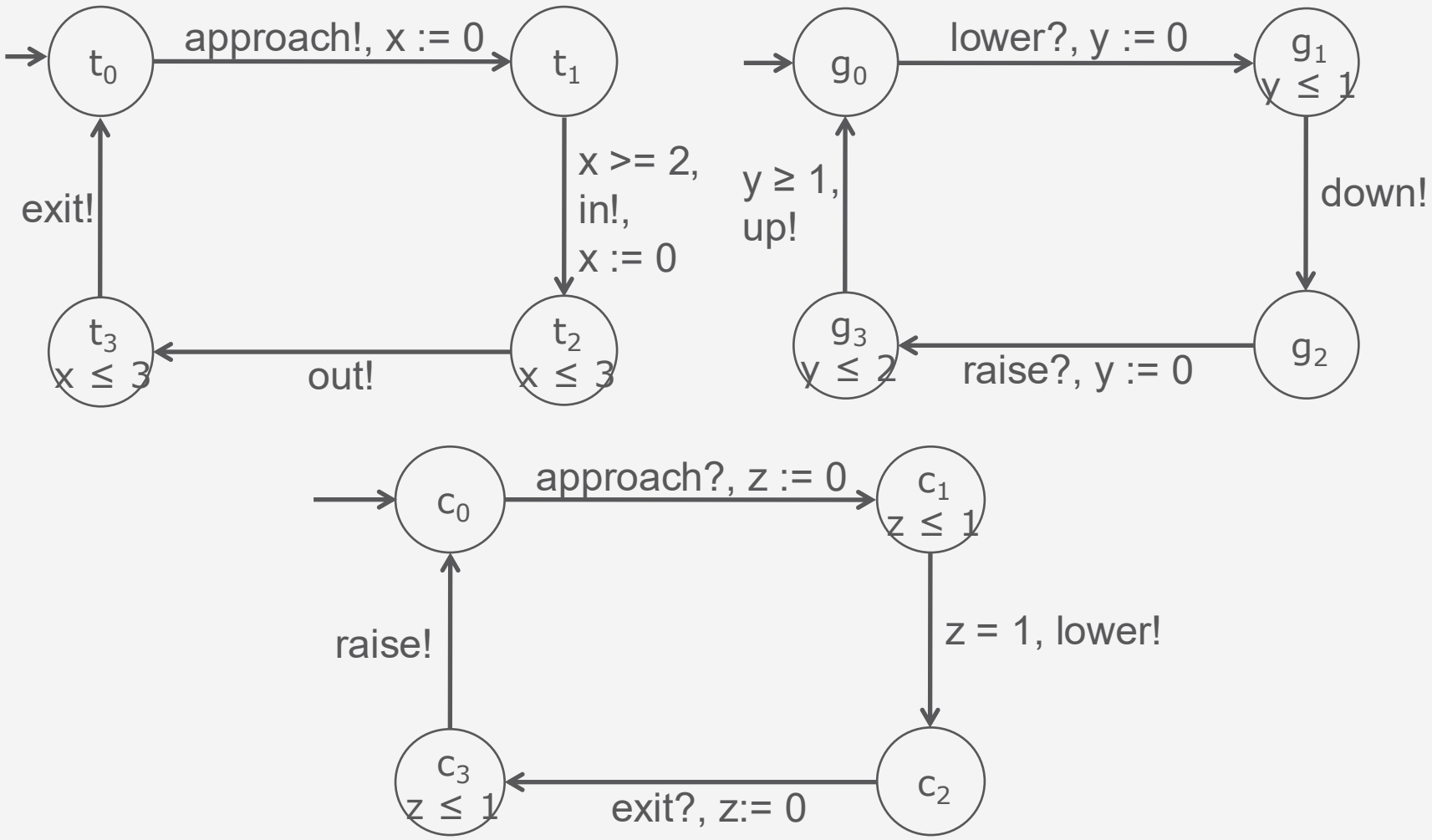
Kontrolleinheit als TA

- Kontrolleinheit (*controller*) sendet genau 1 ZE nach Erhalt des Signals *approach* das Kommando *lower* an die Schranke (*gate*) und das Kommando *raise* nicht mehr als 1 ZE nach Erhalt des Signals *exit*

Modellieren Sie die Kontrolleinheit als Timed Automaton!



Beispiel-Netzwerk

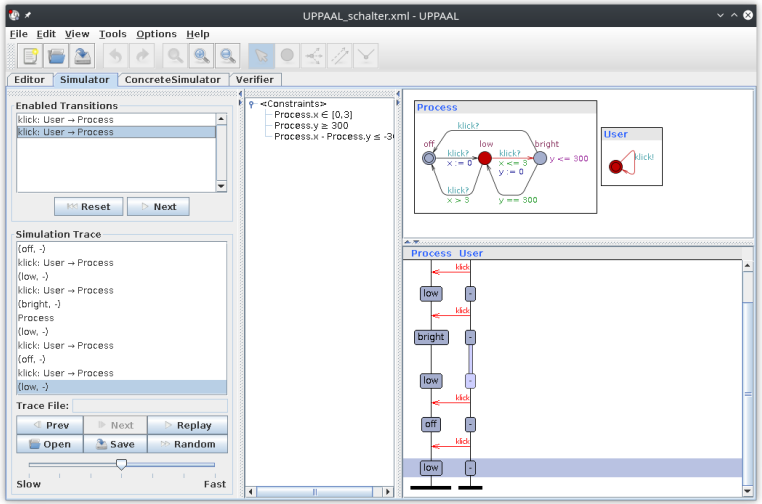


Inhalt

- Einführung
- Semantik
- Netzwerke von Timed Automata
- UPPAAL
- Fazit

UPPAAL

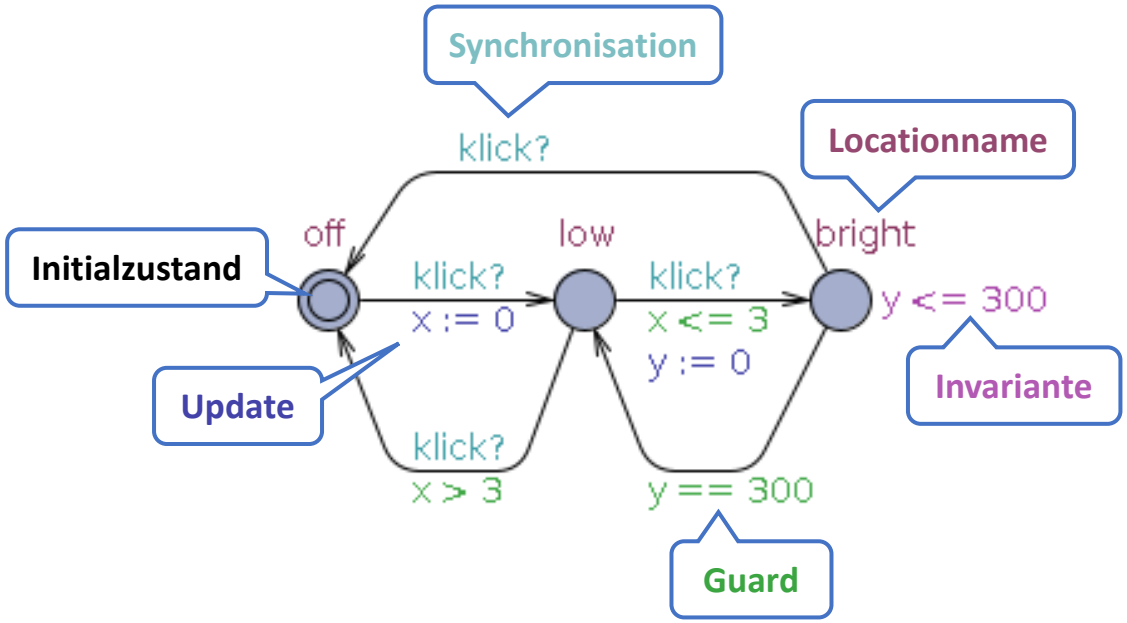
- Tool zur Modellierung, Simulation und Verifikation von TA
- Erste Version 1995, heute in Version 4
- Entwickelt in einer Kooperation von UPPsala (SE) und AALborg (DK) University
- GUI in Java, Algorithmen in C++



Kostenloser Download: www.uppaal.org

UPPAAL Notation

Invarianten, Guards etc. sind farblich kodiert:



UPPAAL Besonderheiten

UPPAAL Timed Automata erweitern klassische TA um zusätzliche Konzepte:

- **Templates** zur Parametrisierung der Automaten
- **Integer-Variablen** und Konstanten, u.a. Arrays (Einschränkung des Wertebereichs möglich)
- **Locationtypen**: *normal, urgent, commited*
- Binäre und **Broadcast Synchronisation**
- **Prioritäten** für Kanäle und Prozesse
- Generierung von **Zufallswerten** (*selections*)

Nicht für Modelchecker verwendbar

Urgent und Commited Locations

U **Urgent**: es darf keine Zeit vergehen.

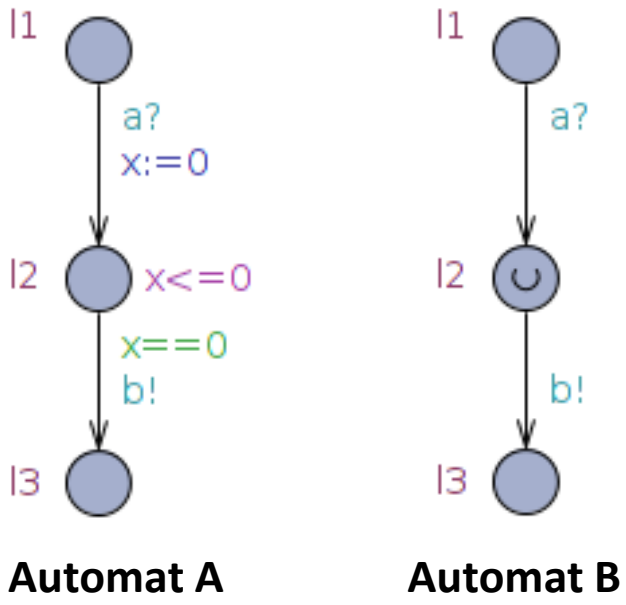
Urgent Locations reduzieren die Anzahl der Uhren in einem Modell und somit die Komplexität der Analyse.

C **Committed**: es darf keine Zeit vergehen. Zusätzlich: es dürfen zunächst keine Automaten weitergehen, die nicht in einer Commited Location sind.

Committed Locations reduzieren Interleaving im Produktautomat (und die Anzahl der Uhren).

Urgent Location: Beispiel

- Eine Nachricht wird auf Kanal a empfangen und sofort auf Kanal b weitergesendet.
- Beide Automaten sind äquivalent.



Urgent Channels

Priorisiert einen Kanal: es darf keine Zeit vergehen, wenn eine Urgent Action-Transition genommen werden kann.

Einschränkung: keine Zeit-Guards erlaubt

Aufbau von UPPAAL-Modellen

Globale Deklarationen

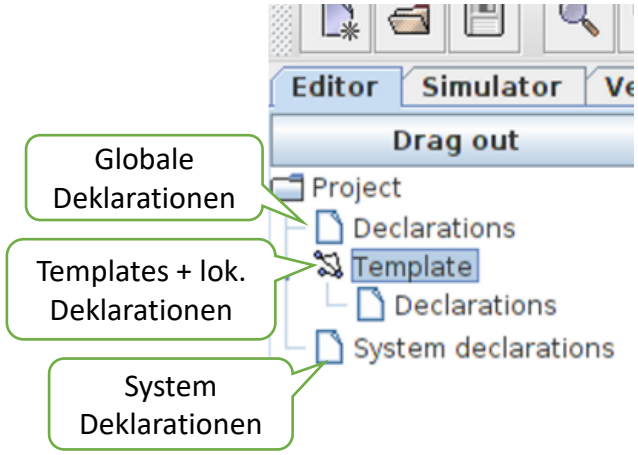
Integer-Variablen, Uhren, Kanäle, Funktionen, Arrays von Variablen, Uhren oder Kanälen

Timed Automata Templates

- Parametrisierte TA, die beliebig oft instanziiert werden können
- Erlaubt mehrere homogene Komponenten

System Deklarationen

- Weitere globale Variablen
- Beschreibung von *Prozessen*: instanziierte TA Templates
- Beschreibung des *Systems*: Parallelkomposition von Prozessen



Fazit

- Timed Automata (TA) erlauben direkte **Spezifikation** von **Zeit und Zeitschranken**
- TA ermöglichen die einfache Spezifikation und Analyse von **Echtzeitsystemen**
- **Parallelkomposition** erlaubt die Spezifikation nebenläufiger Prozesse mit TAs
- Timed Automata können in **UPPAAL** modelliert, simuliert und verifiziert werden